Tashyra Adams

Southern New Hampshire University

CS-499

5 June 2026

5-2 Milestone Four: Enhancement Three: Databases

For this milestone, I focused on enhancing and demonstrating the functionality of my database artifact. The goal was to rebuild the database from my earlier coursework, load new data, apply improvements, and then execute SQL operations that show the database working as intended. This narrative explains the enhancements I made, the SQL statements I executed, and how these changes support the overall functionality of the system. Screenshots are included throughout as evidence of successful execution.

I began by creating a new database environment to house my enhanced version of the original DAD-220 artifact. After creating the database and selecting it for use, I verified that it appeared correctly in MySQL.

```
sql> CREATE DATABASE QuantigrationUpdates;
     USE QuantigrationUpdates;


      OK, 1 row affected in 2.586ms
      OK, 0 records retrieved in 0.534ms
```

```
sql> SHOW Databases;
```

Database
information_schema
mysql
performance_schema
quantigrationupdates
sys

```
OK, 5 records retrieved in 0.962ms
```

Next, I recreated all three tables; Customers, Orders, and RMA; based on the original schema. I then used DESCRIBE statements to confirm that each table was structured correctly with the appropriate fields, data types, and constraints.

```
sql> • DESCRIBE Customers;
```

Field	Type	Null	Key	Default	Extra
CustomerID	int	NO	PRI	NULL	auto_increment
FirstName	varchar(25)	NO		NULL	
LastName	varchar(25)	NO		NULL	
StreetAddress	varchar(50)	NO		NULL	
City	varchar(50)	NO		NULL	
State	char(2)	NO		NULL	
ZipCode	varchar(10)	NO		NULL	
Telephone	varchar(15)	NO	UNI	NULL	

OK, 8 records retrieved in 1.892ms   View: Pages: Edit:

```
sql> • DESCRIBE Orders;
```

Field	Type	Null	Key	Default	Extra
OrderID	int	NO	PRI	NULL	auto_increment
CustomerID	int	NO	MUL	NULL	
SKU	varchar(20)	NO		NULL	
Description	varchar(100)	NO		NULL	

OK, 4 records retrieved in 1.87ms   View: Pages: Edit:

```
sql> • DESCRIBE RMA;
```

Field	Type	Null	Key	Default	Extra
RMAID	int	NO	PRI	NULL	auto_increment
OrderID	int	NO	MUL	NULL	
Step	varchar(50)	NO		NULL	
Status	varchar(20)	NO		NULL	
Reason	varchar(50)	YES		NULL	

OK, 5 records retrieved in 3.702ms   View: Pages: Edit:

After the tables were created, I loaded the newly generated CSV files containing 100 customers, 100 orders, and 100 RMA records. To verify that the data was loaded correctly, I ran SELECT COUNT(*) on each table. All three tables returned a count of 100, confirming that the data import was successful.

```
sql> • SELECT COUNT(*) FROM Customers;
```

COUNT(*)
100

OK, 1 record retrieved in 0.594ms   View: Pages: Edit:

```
sql> • SELECT COUNT(*) FROM Orders;
```

COUNT(*)
100

OK, 1 record retrieved in 1.083ms   View: Pages: Edit:

```
sql> • SELECT COUNT(*) FROM RMA;
```

COUNT(*)
100

OK, 1 record retrieved in 0.503ms   View: Pages: Edit:

One enhancement I made to the Customers table was adding an Email column. This improves the usefulness of the customer data and aligns the table with more realistic business requirements. After adding the column, I used it in my INSERT and UPDATE demonstrations.

To demonstrate full CRUD functionality and relational integrity, I executed several SQL operations: INSERT, UPDATE, DELETE, verification, and a multi-table JOIN.

I inserted a new test customer into the Customers table using all required fields, including the enhanced Email column. The database confirmed that one row was successfully added.

```
sql> • INSERT INTO Customers
       (CustomerID, FirstName, LastName, StreetAddress, City, State, ZipCode, Telephone, Email)
       VALUES
       (101, 'TestFirst', 'TestLast', '123 Test St', 'TestCity', 'OH', '44087', '330-555-1234', 'test101@examp

        OK, 1 row affected in 2.953ms
```

Next, I updated the Email field for the newly inserted customer. This demonstrates the ability to modify existing records.

```
sql> • UPDATE Customers
       SET Email = 'updated101@example.com'
       WHERE CustomerID = 101;

        OK, 1 row affected in 6.852ms
```

After verifying the update, I deleted the test customer from the table. MySQL confirmed that one row was removed.

```
sql> • DELETE FROM Customers
       WHERE CustomerID = 101;

        OK, 1 row affected in 3.951ms
```

To confirm the deletion, I ran a SELECT query filtering by the CustomerID. The result returned zero rows, proving the delete was successful.

```
sql> • SELECT * FROM Customers WHERE CustomerID = 101;
```

CustomerID	FirstName	LastName	StreetAddress	City	State	ZipCode	Telephone	Email

OK, 0 records retrieved in 1.117ms   View: Pages: Edit:

To show that the relationships between Customers, Orders, and RMA were functioning correctly, I ran a JOIN query that combined all three tables. This query returned customer information alongside their orders and RMA statuses, demonstrating referential integrity and proper table linkage.

```sql
sql> • SELECT
        Customers.CustomerID,
        Customers.FirstName,
        Customers.LastName,
        Orders.OrderID,
        Orders.SKU,
        RMA.RMAID,
        RMA.Status
    FROM Customers
    JOIN Orders ON Customers.CustomerID = Orders.CustomerID
    JOIN RMA ON Orders.OrderID = RMA.OrderID
    LIMIT 20;
```

CustomerID	FirstName	LastName	OrderID	SKU	RMAID	Status
1	Customer1	Last1	1	SKU001	1	Pendin
2	Customer2	Last2	2	SKU002	2	Awaiti
3	Customer3	Last3	3	SKU003	3	Comple
4	Customer4	Last4	4	SKU004	4	Reject
5	Customer5	Last5	5	SKU005	5	Pendin
6	Customer6	Last6	6	SKU006	6	Awaiti
7	Customer7	Last7	7	SKU007	7	Comple
8	Customer8	Last8	8	SKU008	8	Reject
9	Customer9	Last9	9	SKU009	9	Pendin
10	Customer10	Last10	10	SKU010	10	Awaiti
11	Customer11	Last11	11	SKU011	11	Comple

OK, 20 records retrieved in 2.009ms   View: Pages: Edit:

This milestone allowed me to rebuild and enhance my original database artifact while demonstrating that it functions correctly through SQL operations. Adding the Email field improved the structure of the Customers table, and executing INSERT, UPDATE, DELETE, and JOIN queries provided clear evidence that the database behaves as expected. These enhancements strengthen the artifact and prepare it for integration into the final project.